Instituto Infnet

Projeto de Bloco: Inteligência Artificial Aplicada

Radar Solar Municipal

Geração distribuída de energia no Brasil por município

Teste de Performance 3

Rafael Celestino Tomiazi
Professor Renan Silva Santos

19 de outubro de 2026

Repositório no GitHub:
https://github.com/RafaelTomiazi/radar-solar-municipal

App publicado:
https://radar-solar-municipal-h3abtstmymwssr8a9wpb9m.streamlit.app/

Ferramenta de IA utilizada no desenvolvimento: Claude (Anthropic)

O que mudou no TP3

No TP2 o painel era um arquivo só, com cinco abas, lendo CSV direto. No TP3 ele virou uma aplicação com oito páginas e menu de navegação, ganhou uma API própria feita com FastAPI, que o painel consome por HTTP, e uma fonte nova coletada com Selenium: a geração do país por fonte, da página Energia Agora do ONS. Também revisei o Project Charter e o Data Summary Report e planejei o uso de LLM para o TP4.

Onde está cada item da rubrica

Item da rubrica	Onde está	Evidência
Menu de navegação com múltiplas páginas	`app.py` (`st.navigation`) e pasta `paginas/`	Figura 1. Oito páginas em três seções: Análise, Dados coletados e Projeto
API desenvolvida com FastAPI	`Code/API/api.py`	Quatro rotas GET e uma POST, Swagger em `/docs` (Figura 5)
Projeto integrado com a API	`paginas/5_api.py` e `Code/App/comum.py`	A página chama a API por HTTP (Figuras 3 e 4). O `streamlit run app.py` sobe a API junto
Mais dados coletados com Beautiful Soup	`Code/DataAcquisition/scraping_noticias.py`	83 notícias da ANEEL com texto completo, página Notícias ANEEL
Selenium para adquirir dados de sites	`Code/DataAcquisition/scraping_ons_selenium.py`	`data/external/geracao_ons.csv`, página Geração agora (Figura 2)
Documentação atualizada	`Docs/Project/project_charter.md` e `Docs/DataReport/data_summary_report.md`	Seções 9 e 10 do Charter, fonte ONS e API no Data Summary
Planejamento de IA via LLM	`paginas/8_proximos_passos.py` e seção 10 do Charter	Figura 6. Quatro tarefas com dado de entrada, motivo e forma de implementar

Testes que fiz antes de entregar

Visitei as oito páginas num navegador automatizado e nenhuma levantou exceção. Testei a API pelo terminal e pela página: ranking, ficha de município, POST com população, código inexistente (404) e população negativa (422). O teste da API pegou um erro: os 4 municípios do Amazonas sem nenhum empreendimento davam erro 500, porque região e média vinham vazias. Corrigi na carga da API.

Seguindo a correção do TP1, instalei o `requirements.txt` num ambiente virtual limpo, rodei `pip check` (sem conflitos) e abri o app por esse ambiente.

Tive um problema com a estrutura de páginas que vale registrar: com a pasta chamada `pages/`, o Streamlit ativa o modo antigo de multipágina e roda cada arquivo direto, sem passar pelo `app.py`, então o menu e os filtros sumiam. Renomeei para `paginas/`.

Como rodar

```
git clone https://github.com/RafaelTomiazi/radar-solar-municipal.git
cd radar-solar-municipal
python -m venv .venv
source .venv/bin/activate        (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
streamlit run app.py
```

O app sobe a API na porta 8000 se ela não estiver rodando. Para rodar a API separada: `uvicorn Code.API.api:app --reload --port 8000`. Para coletar de novo a geração do ONS: `python Code/DataAcquisition/scraping_ons_selenium.py` (precisa do Chrome instalado).

Aplicação

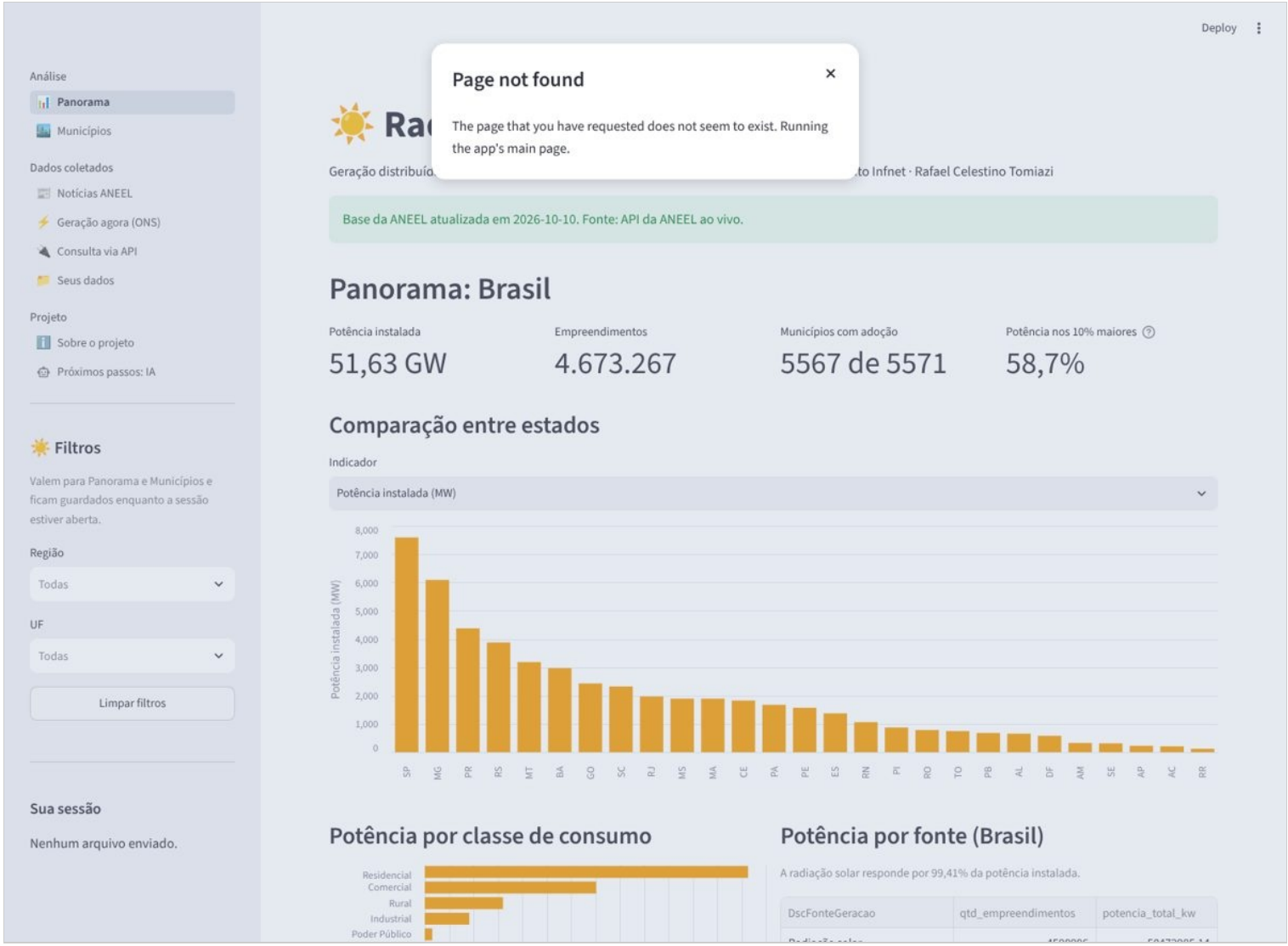


Figura 1. Menu de navegação com as oito páginas e os filtros na barra lateral.

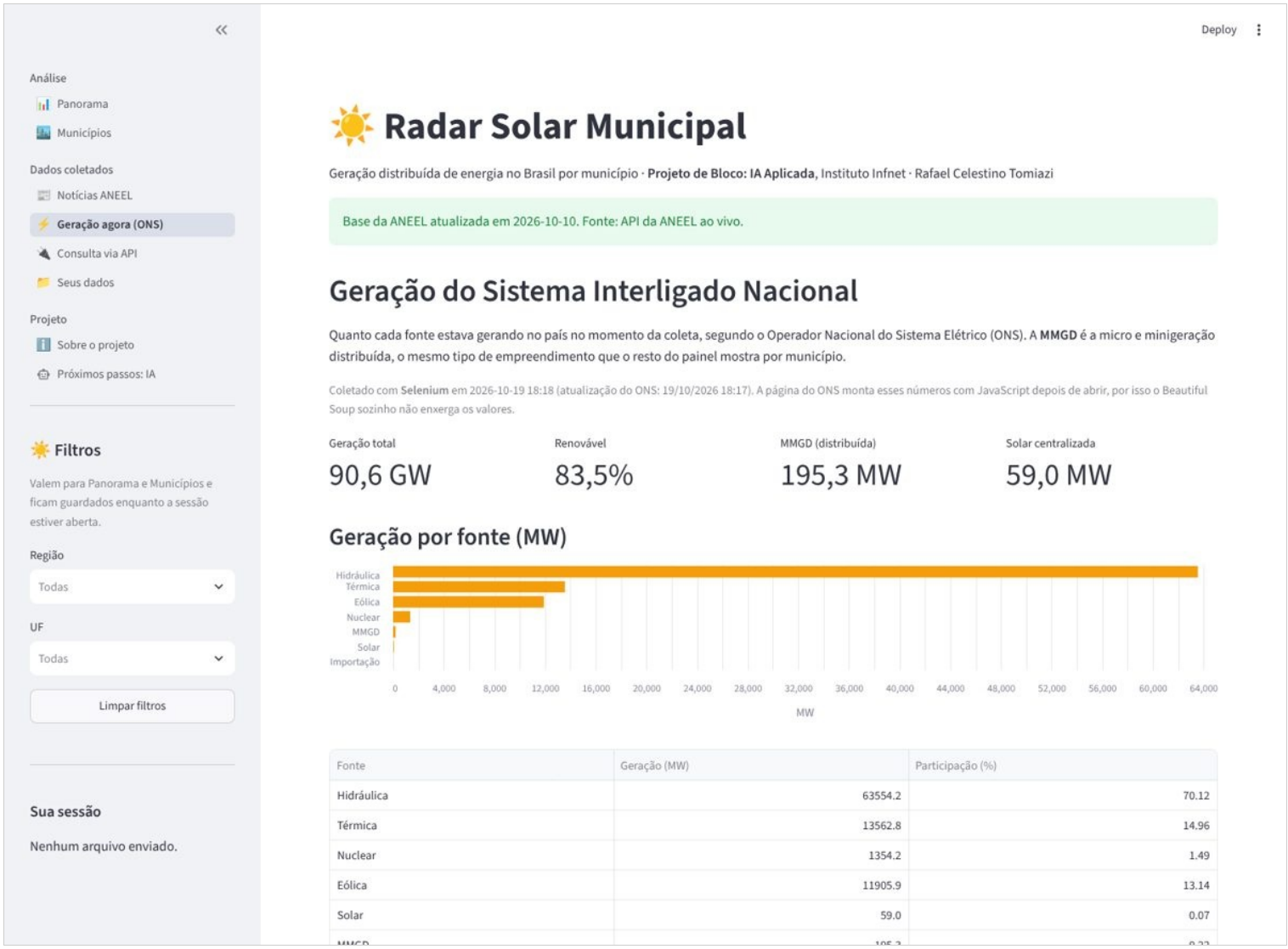


Figura 2. Geração do país por fonte, coletada do ONS com Selenium.

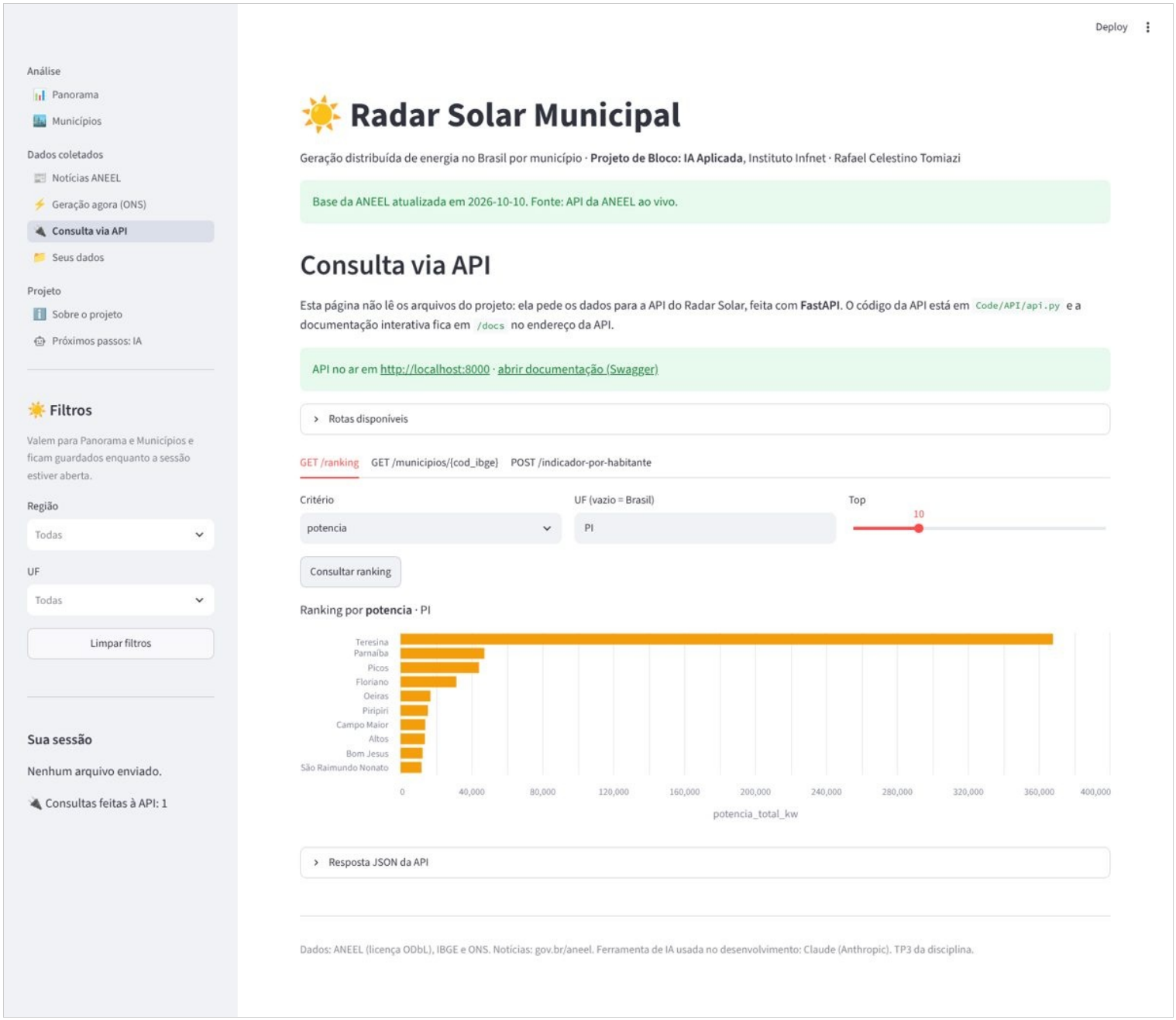


Figura 3. Página Consulta via API: GET /ranking com filtro por UF.

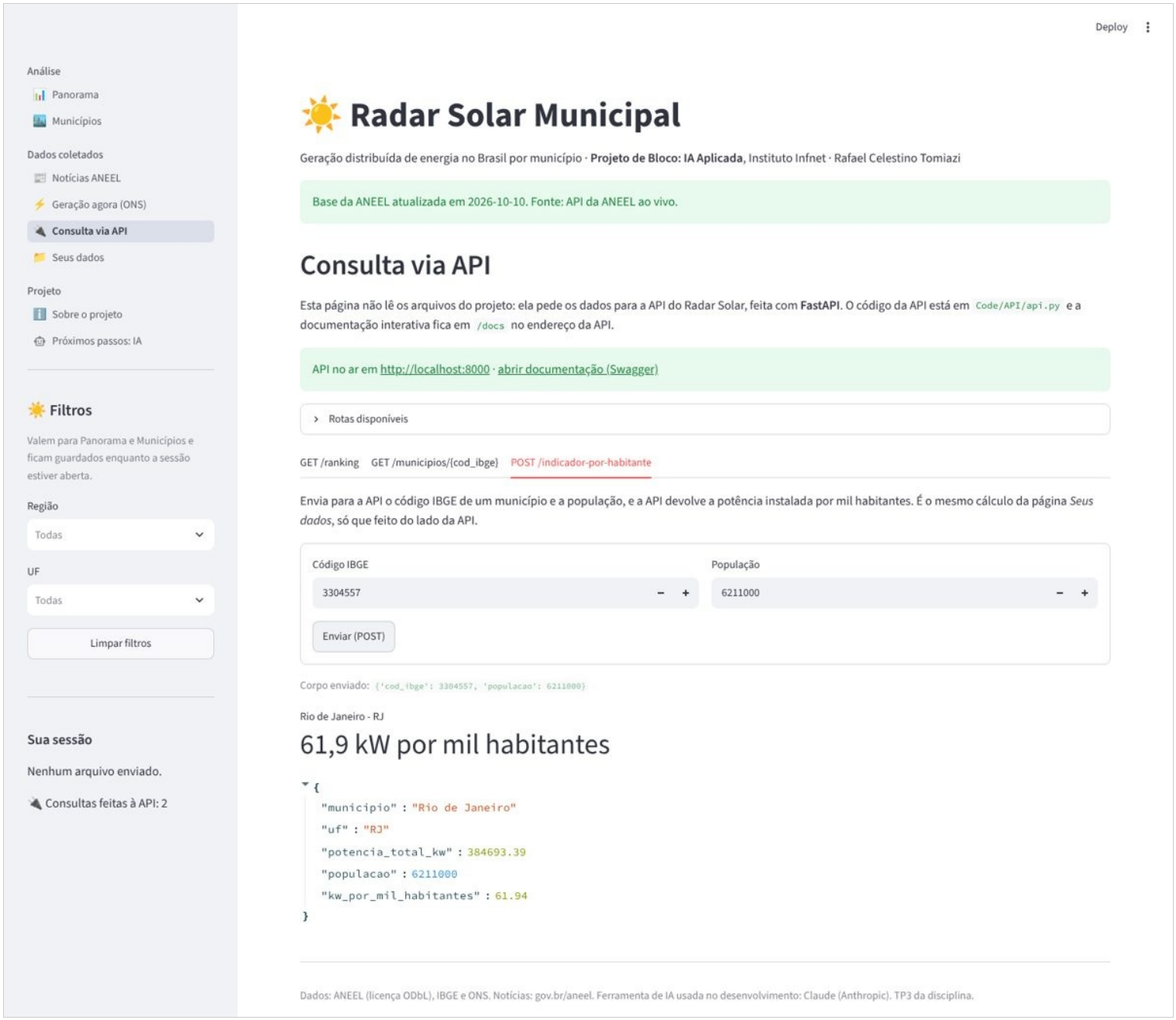


Figura 4. POST /indicador-por-habitante e a resposta JSON da API.

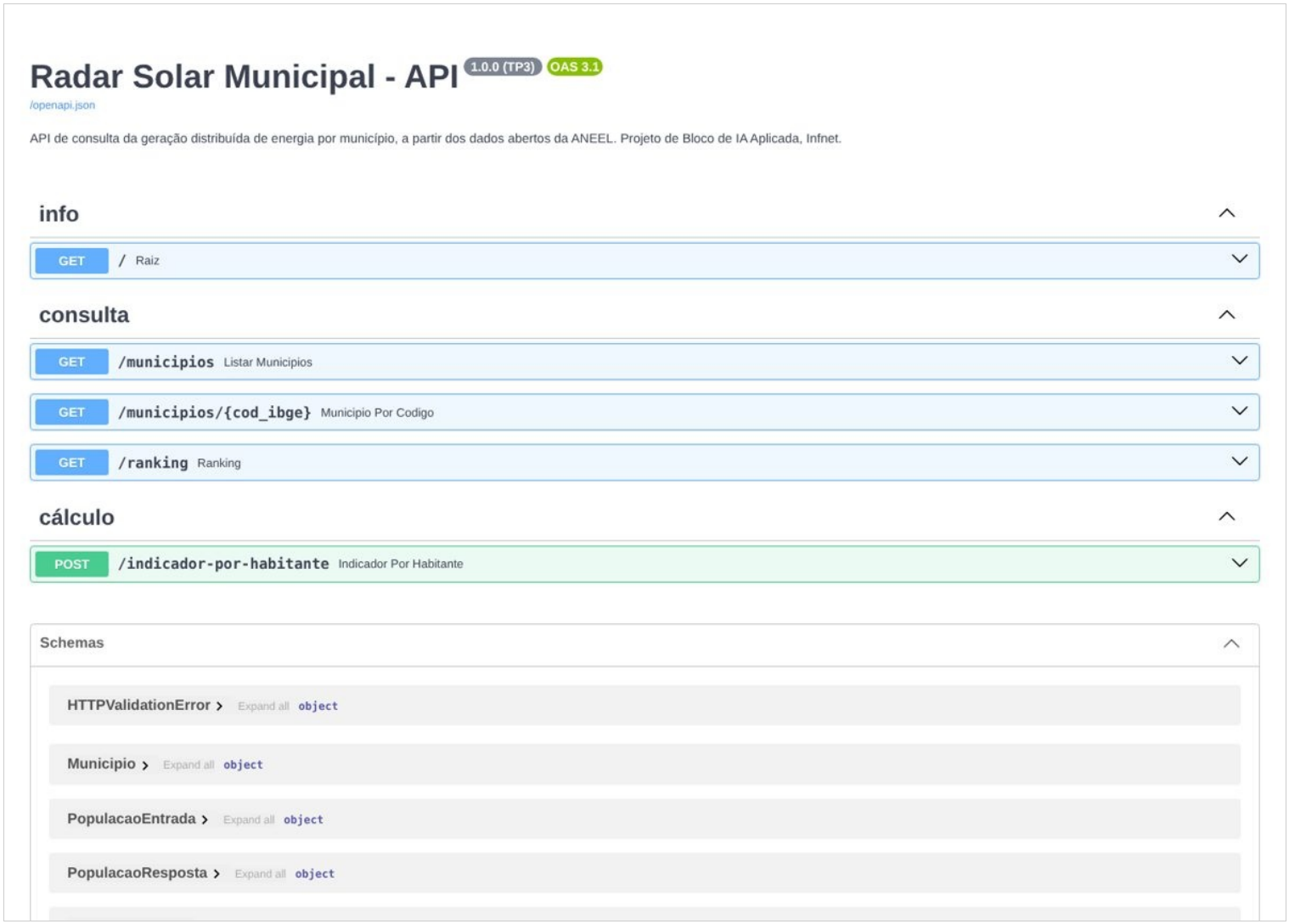


Figura 5. Documentação automática da API (Swagger) em /docs.

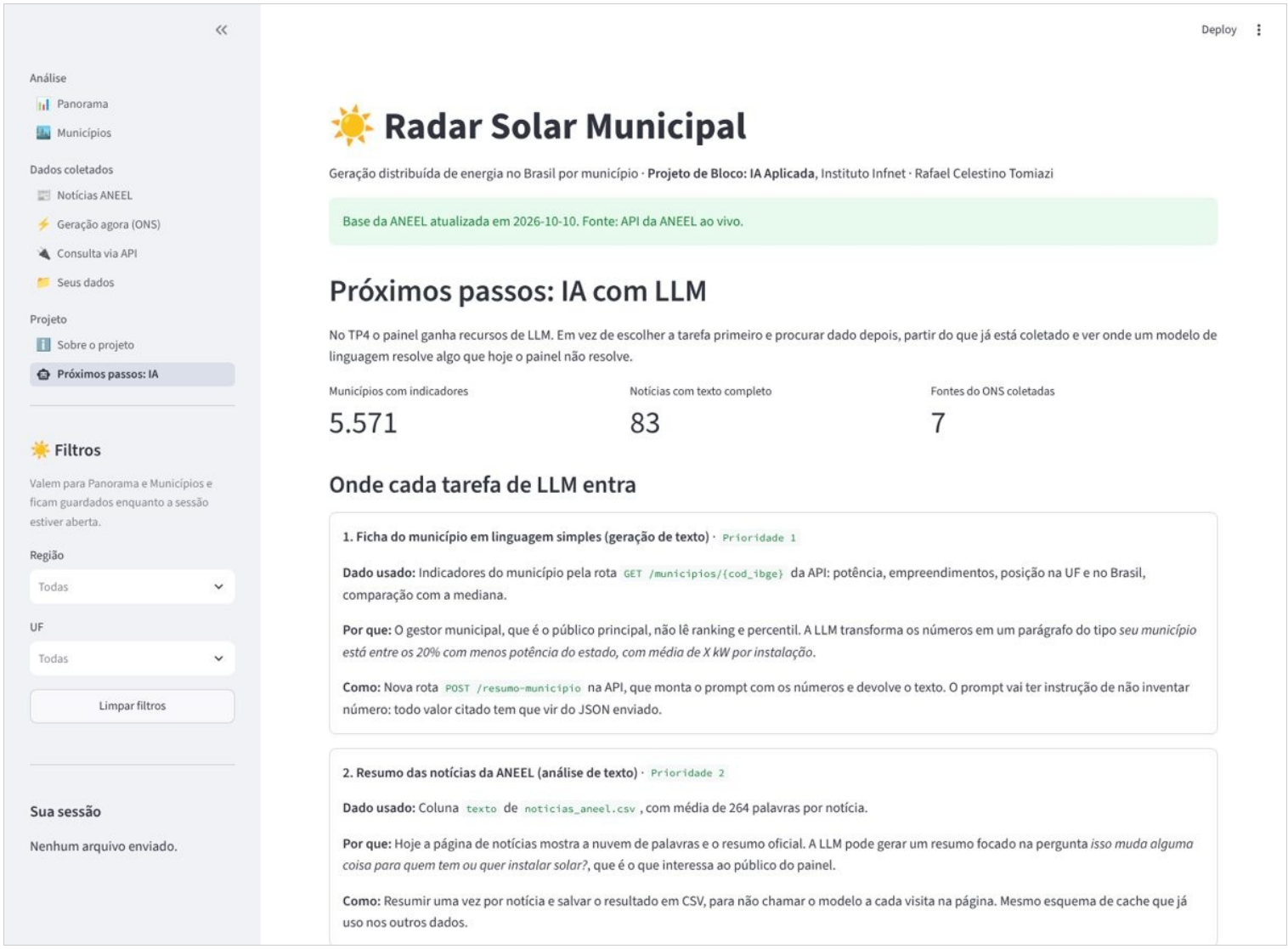


Figura 6. Página de planejamento do uso de LLM no TP4.

Project Charter

Projeto: Radar Solar Municipal **Autor:** Rafael Celestino Tomiazi **Disciplina:** Projeto de Bloco, Inteligência Artificial Aplicada, Instituto Infnet
Professor: Renan Silva Santos **Data:** 24 de agosto de 2026 (TP1), atualizado em 21 de setembro de 2026 (TP2) e em 19 de outubro de 2026 (TP3)

1. Problema de negócio

A ANEEL publica a lista completa dos empreendimentos de micro e minigeração distribuída do Brasil. São 4,67 milhões de registros, 51,6 GW de potência instalada e 99,41% disso vem de energia solar.

O dado é aberto, mas vem em um arquivo Parquet de 101 MB com 31 colunas de nome técnico. Quem consegue abrir esse arquivo já trabalha com dados. O gestor municipal que precisaria da informação para justificar uma política de incentivo normalmente não tem equipe para isso.

Quando processei a base eu esperava achar municípios sem nenhuma adoção. Achei só quatro. Então o problema não é falta de adoção, é diferença de intensidade: os 10% de municípios com maior potência instalada concentram 58,7% do total do país, e o município mediano tem 3 MW contra 19,7 MW do município no percentil 90.

2. Objetivos

Objetivo geral: transformar a base bruta da ANEEL em um painel que mostre como a geração solar distribuída está espalhada pelo território brasileiro.

Objetivos específicos:

- Consolidar os 4,67 milhões de registros em indicadores por município, sem perder registro no caminho
- Incluir também os municípios que não têm nenhum empreendimento, usando a lista oficial do IBGE
- Mostrar os indicadores em um painel que qualquer pessoa consiga abrir e entender
- Manter o projeto reprodutível, de forma que outra pessoa consiga rodar tudo a partir do repositório
- Deixar o gestor explorar o dado sozinho, escolhendo região, UF, município e indicador, sem depender de alguém para gerar o recorte (TP2)
- Trazer o contexto regulatório para dentro do painel, com as notícias publicadas pela ANEEL (TP2)
- Permitir que o usuário complemente o painel com os dados que ele já tem, como população ou orçamento de um programa, e baixe o resultado (TP2)
- Disponibilizar os indicadores municipais por uma API, para que outro sistema, ou a própria camada de IA do TP4, consulte os dados sem depender dos arquivos (TP3)
- Colocar a GD instalada em perspectiva com o que o país está gerando, usando a geração por fonte do ONS (TP3)
- Organizar o painel em páginas separadas por assunto, com menu de navegação (TP3)

3. Escopo

Está no escopo: coleta automatizada na API da ANEEL e na API do IBGE, processamento que agrega os registros individuais em indicadores municipais, e um painel em Streamlit apresentando o resultado.

No TP2 o escopo passou a incluir também:

- extração das notícias do site da ANEEL com Beautiful Soup, com o texto completo de cada notícia salvo em CSV e TXT, e a exibição desse conteúdo no painel (nuvem de palavras, termos mais frequentes, notícias por categoria e por mês)
- interface interativa, com filtros de região e UF, busca de município, ranking configurável e comparação entre municípios
- upload de CSV pelo usuário, validado e combinado com a base pelo código IBGE, e download dos dados filtrados e combinados
- cache e estado de sessão, para o painel responder rápido e não perder as escolhas do usuário entre uma aba e outra
- estimativa de população do IBGE (API SIDRA), usada como exemplo de upload para calcular a potência por mil habitantes
- publicação do painel no Streamlit Community Cloud, com o código versionado no GitHub

No TP3 entraram:

- aplicação com múltiplas páginas e menu de navegação (`st.navigation`), com os filtros na barra lateral valendo para todas as páginas
- API com FastAPI (`Code/API/api.py`) com quatro rotas GET de consulta e uma rota POST de cálculo, documentada no Swagger em `/docs`, e uma página do app que consome essa API por HTTP
- extração da geração do Sistema Interligado Nacional por fonte na página "Energia Agora" do ONS, com Selenium, porque essa página monta os números por JavaScript
- planejamento das tarefas com LLM para o TP4, com uma página no app explicando cada uma

Está fora do escopo: previsão de adoção futura, dados de irradiação solar por coordenada (dependem de fonte paga), informação individual do titular do empreendimento e cálculo de retorno financeiro de instalação.

4. Metas

Indicador	Meta
Municípios cobertos pelo painel	Os 5.571 do IBGE
Registros perdidos no processamento	Zero
Colunas com dado pessoal no arquivo processado	Zero
Tempo de carregamento do painel	Abaixo de 3 segundos
Data de referência do dado visível na tela	Sempre
Notícias da ANEEL com texto completo extraído	Todas as publicadas na listagem pública
Arquivo enviado pelo usuário que quebra o painel	Nenhum: arquivo inválido gera mensagem de erro, não exceção
Painel acessível sem instalar nada	Sim, publicado no Streamlit Community Cloud
Instalação do zero com `pip install -r requirements.txt`	Sem conflito de dependências
Rotas da API respondendo com dado válido para os 5.571 municípios	Todas (TP3). Código inexistente devolve 404, dado inválido devolve 422, nunca erro 500
Passos para ver a API funcionando	Um só: `streamlit run app.py` já sobe a API junto (TP3)
Página que depende de dado coletado fora do ar	Mostra aviso com o comando para coletar, não quebra o app (TP3)

5. Público-alvo e stakeholders

O público principal são os gestores de secretarias municipais de meio ambiente e de desenvolvimento econômico, principalmente de cidades pequenas e médias que não têm equipe de análise de dados. É quem precisa justificar uma decisão de incentivo e hoje não consegue abrir a base.

Como público secundário eu penso em cooperativas de energia e empresas integradoras, que avaliam onde vale expandir, e em pesquisadores e jornalistas que cobrem transição energética.

Stakeholder	Papel
Secretarias municipais	Usuário principal
Cooperativas e integradores solares	Usuário secundário
Pesquisadores e jornalistas de dados	Usuário secundário
ANEEL e IBGE	Fornecedores dos dados (base de GD, notícias, municípios e população)
Instituto Infnet	Avaliador
Rafael Tomiazi	Desenvolvimento do projeto

6. ESG e ODS

O projeto fica no pilar Ambiental, com um componente de Governança por usar dado público e tratar com cuidado as colunas de dado pessoal.

O ODS principal é o **7, Energia Limpa e Acessível**. O painel atua sobre a meta 7.2, que trata de aumentar a participação de renováveis na matriz, dando visibilidade a como essa participação está distribuída no território.

Como secundários eu vejo o **ODS 11, Cidades e Comunidades Sustentáveis**, pelo recorte municipal, o **ODS 13, Ação Contra a Mudança Global do Clima**, pela troca de fonte fóssil por renovável, e o **ODS 10, Redução das Desigualdades**, por causa da concentração de 58,7% da potência em 10% dos municípios.

7. Metodologia

Uso o TDSP para organizar o projeto e as pastas do repositório, e o CRISP-DM para orientar o ciclo analítico dentro da fase de dados. Os dois se encaixam bem: o TDSP diz onde cada artefato mora e o CRISP-DM diz como ir do dado bruto ao resultado.

As fases do TDSP aplicadas ao projeto:

Fase do TDSP	Como se aplica aqui	Artefato	Situação
Business Understanding	Definição do problema, metas, público-alvo e ODS	Project Charter	Feito
Data Acquisition & Understanding	Coleta nas APIs da ANEEL e do IBGE, scraping das notícias da ANEEL (Beautiful Soup), geração do ONS (Selenium) e população do SIDRA	Data Summary Report	Atualizado no TP3
Modeling	Engenharia dos atributos municipais a partir dos registros individuais. No TP4, prompts e uso de LLM sobre esses atributos	Model Report	Planejado (página Próximos passos: IA)
Deployment	Publicação do painel em Streamlit e da API com FastAPI	`app.py`, `paginas/`, `Code/API/api.py`	Terceira versão: multipágina com API
Customer Acceptance	Validação do painel com leitores fora da área técnica	Project Final Report	A fazer

O CRISP-DM aparece dentro das duas fases de dados no ciclo de entender o negócio, entender o dado, preparar o dado e avaliar o resultado. Foi esse ciclo que me fez trocar a hipótese inicial do projeto: eu entrei querendo mapear vazios de adoção, olhei o dado, vi que quase não existem vazios, e voltei para redefinir a pergunta em cima da intensidade.

8. Riscos

A ANEEL parou de atualizar o dataset entre setembro e novembro de 2025 por causa da migração do sistema SISGD, então a fonte pode ficar parada por um tempo grande. Por isso o painel sempre mostra a data de referência do dado e de onde ele veio.

O arquivo bruto tem 101 MB e passa do limite de 100 MB por arquivo do GitHub. Deixei ele no `.gitignore` e o script de coleta reconstrói quando precisa.

Se a API da ANEEL estiver fora no momento em que alguém abrir o painel, o app cai em um cache local e avisa na tela que o dado é guardado.

O scraping depende do HTML do site da ANEEL. Se o layout mudar, o seletor para de achar as notícias. O script foi feito para parar com erro nesse caso, em vez de gravar um arquivo vazio por cima do que já existe, e o app continua funcionando com a última coleta versionada.

Durante o período eleitoral o site da ANEEL deixa públicas só as notícias recentes, e o arquivo por ano pede login. Por isso a coleta atual tem 83 notícias, a maior parte de julho a setembro de 2026, e só 5 delas falam de geração distribuída ou solar. Depois do período eleitoral vale rodar o scraping de novo para ampliar a base.

O arquivo enviado pelo usuário pode vir em formato diferente do esperado (separador, codificação, código IBGE com erro). O app valida o arquivo, descarta as linhas com código inválido ou repetido e mostra um aviso com a quantidade descartada.

O Selenium depende de ter o Chrome instalado na máquina que roda a coleta, e a página do ONS pode mudar o texto ou demorar para carregar. O script espera até 20 segundos pelos números e, se não aparecerem, para com erro sem gravar nada. O app mostra a última coleta versionada. A coleta também é uma fotografia de um momento: à noite a solar quase não gera, então o número depende da hora em que o script rodou, e a página deixa isso explícito.

A API e o app rodam juntos. Se a API não estiver no ar, o app tenta subir ela numa thread. Se mesmo assim ela não responder, só a página Consulta via API para de funcionar, com uma mensagem dizendo como subir a API, e o resto do painel segue lendo os arquivos.

Testando a API eu achei os 4 municípios do Amazonas sem nenhum empreendimento devolvendo erro 500, porque vinham com região e média vazias e valor vazio não vira JSON. Corrigi preenchendo a região pela UF e a média com zero na carga da API.

9. Revisão do problema no TP3

O problema de negócio continua o mesmo: o dado de geração distribuída é aberto mas inacessível para o gestor municipal, e a desigualdade está na intensidade da adoção, não na ausência. O que mudou com as ferramentas novas foi o alcance da solução.

Com a FastAPI, o painel deixou de ser a única porta de entrada para o dado. Um sistema da prefeitura, uma planilha ou um script consegue pedir a ficha de um município por HTTP. Isso também resolve um problema que eu teria no TP4: a LLM vai precisar consultar números exatos, e é mais seguro ela pedir para uma rota que devolve o valor do que eu colar a base inteira no prompt.

Com o Selenium, entrou uma pergunta que o painel não respondia: quanto da GD instalada está gerando de fato. A base da ANEEL diz quanto foi instalado, e o ONS diz quanto o país está gerando agora por fonte, com a MMGD separada. Na coleta de 19/10/2026 às 18h17 a MMGD gerava 195 MW, contra mais de 50 GW instalados, porque era fim de tarde. Uma coleta só não fecha conclusão, mas mostra que a métrica de potência instalada sozinha superestima a contribuição real da GD ao longo do dia.

Ajustei as metas na tabela da seção 4 para incluir a API e o comportamento das páginas novas quando falta dado.

10. Planejamento do uso de LLM (TP4)

Parti do que já está coletado para escolher as tarefas, em ordem de prioridade. O detalhe de cada uma está na página Próximos passos: IA do app.

Tarefa	Dado de entrada	Para quê
Ficha do município em linguagem simples (geração de texto)	Rota `GET /municipios/{cod_ibge}` da API	Transformar posição, percentil e mediana em um parágrafo que o gestor entenda
Resumo das notícias da ANEEL (análise de texto)	Coluna `texto` das 83 notícias	Dizer em poucas linhas se a notícia muda algo para quem tem ou quer instalar solar
Classificação do impacto para GD (classificação)	As mesmas notícias	Substituir a marcação por palavra-chave, que acha só 5 de 83 notícias, por três classes: afeta GD, afeta consumidor em geral, não afeta
Perguntas e respostas (Q&A)	Rotas da API como ferramentas do modelo	Responder perguntas como "5 municípios do Piauí com mais potência" sem mexer em filtro. Fica por último por ser o mais arriscado

Em todas, o número citado no texto gerado tem que vir do dado enviado ao modelo, a resposta aparece marcada como gerada por IA, a chave da API do modelo fica fora do código e as respostas ficam em cache por município e por notícia.

Data Summary Report

Projeto: Radar Solar Municipal **Autor:** Rafael Celestino Tomiazi **Data:** 24 de agosto de 2026 (TP1), atualizado em 21 de setembro de 2026 (TP2) e em 19 de outubro de 2026 (TP3)

O Data Summary Report é o documento da fase de Data Acquisition & Understanding do TDSP. A primeira versão, do TP1, cobria a base da ANEEL e a malha do IBGE. No TP2 entraram as notícias do site da ANEEL, extraídas com Beautiful Soup, a estimativa de população do IBGE e os arquivos que o próprio usuário envia pelo painel. No TP3 entrou a geração por fonte do ONS, extraída com Selenium de uma página dinâmica, e a API do próprio projeto, que passou a ser uma forma de acessar os dados. Documentei só as colunas que eu uso de fato, não as 31 do arquivo original da ANEEL.

1. Fontes de dados

ANEEL, empreendimentos de geração distribuída

Acesso pela API do CKAN em `https://dadosabertos.aneel.gov.br/api/3/action/package_show`, sem chave. Arquivo Parquet de 101 MB, 4.673.268 linhas e 31 colunas. Licença Open Data Commons ODbL, atualização verificada em 21/08/2026.

É a fonte principal. Cada linha é um empreendimento de micro ou minigeração ligado à rede, com localização, fonte de energia, classe de consumo e potência instalada.

Consulto a API em vez de fixar o link do arquivo porque a URL direta muda quando a ANEEL republica o dataset.

IBGE, malha de municípios

Acesso em `https://servicodados.ibge.gov.br/api/v1/localidades/municipios`, JSON, sem chave. Devolve os 5.571 municípios do Brasil com código oficial, nome e hierarquia territorial.

Uso como referência territorial. É ela que garante que município sem nenhum empreendimento apareça no painel, porque na base da ANEEL só existe linha para quem tem instalação.

ANEEL, notícias do site (novo no TP2)

Extração com Beautiful Soup em `https://www.gov.br/aneel/pt-br/assuntos/noticias`, página pública, sem chave. O script `Code/DataAcquisition/scraping_noticias.py` percorre a listagem, que vem paginada de 30 em 30 pelo parâmetro `b_start:int`, pega data, categoria, título, resumo e link de cada notícia e depois entra em cada uma para ler o texto completo. Entre as requisições há uma pausa para não sobrecarregar o site. O `robots.txt` do gov.br não bloqueia essa área.

A coleta de 21/09/2026 trouxe 83 notícias, publicadas entre 15/03/2022 e 18/09/2026, em 20 categorias, com média de 264 palavras por notícia.

IBGE, estimativa de população (novo no TP2)

Acesso pela API SIDRA em `https://apisidra.ibge.gov.br/values/t/6579/n6/all/v/9324/p/last`, JSON, sem chave. Tabela 6579, variável 9324 (população residente estimada), todos os municípios, último ano disponível. Traz os 5.571 municípios com a estimativa de 2026, que soma 214,2 milhões de habitantes. O script é o `Code/DataAcquisition/coleta_populacao.py`.

ONS, geração por fonte em tempo real (novo no TP3)

Extração com Selenium em `https://www.ons.org.br/paginas/energia-agora/carga-e-geracao`, página pública, sem chave. O script é o `Code/DataAcquisition/scraping_ons_selenium.py`.

Essa é a fonte que justifica o Selenium. Testei primeiro com requests e Beautiful Soup: a página devolve 154 KB de HTML, mas os valores de geração não estão nele, porque o JavaScript da página busca os números e monta o quadro depois que ela abre. O Selenium abre a página num Chrome sem janela, espera aparecer algum valor em MW (checa de 2 em 2 segundos, até 20 segundos) e lê o texto da tela. Os valores vêm em formato brasileiro (63.554,2 MW) e são convertidos para número.

O quadro traz a geração do Sistema Interligado Nacional separada em hidráulica, térmica, nuclear, eólica, solar, MMGD (micro e minigeração distribuída) e importação. A MMGD é o mesmo tipo de empreendimento que a base da ANEEL lista por município, então essa fonte liga o que está instalado com o que está gerando.

A coleta de 19/10/2026 às 18h17 trouxe 90,6 GW de geração total, 83,5% renovável, com a MMGD gerando 195,3 MW e a solar centralizada 59,0 MW.

API do Radar Solar (novo no TP3)

Não é uma fonte externa, é a forma nova de acessar os dados processados. A API em `Code/API/api.py` lê o `data/processed/cobertura_municipios.csv` quando sobe e serve esses dados por HTTP. A página Consulta via API do app usa essa API em vez de ler o CSV. No TP4 ela vai ser o caminho pelo qual a LLM consulta números exatos.

Dados enviados pelo usuário (novo no TP2)

Pela aba Seus dados o usuário envia um CSV com o código IBGE do município e uma ou mais colunas numéricas, como população, número de escolas ou orçamento de um programa. O arquivo não é gravado no servidor: ele fica só na sessão do usuário, é validado e combinado com a base da ANEEL pelo código IBGE.

Objetivo de uso de cada fonte

Fonte	Objetivo de uso no projeto	Onde aparece no painel
ANEEL, empreendimentos de GD	Medir a adoção da geração distribuída por município: quantidade, potência total e potência média	Panorama e Municípios
IBGE, malha de municípios	Garantir os 5.571 municípios no painel, inclusive os sem instalação, com nome oficial, UF e região	Panorama, Municípios e filtros
API da ANEEL (metadados)	Mostrar a data de referência da base, para o usuário nunca ver um número sem saber de quando ele é	Topo do painel
ANEEL, notícias	Dar o contexto regulatório: o que a agência está discutindo (tarifas, leilões, consultas públicas, regras de GD) e como isso muda com o tempo	Notícias ANEEL
IBGE, população	Transformar potência absoluta em indicador relativo (kW por mil habitantes), que compara municípios de tamanhos diferentes	Exemplo pronto de upload
Arquivo do usuário	Deixar o gestor cruzar o dado da ANEEL com a informação que ele já tem sobre o município	Seus dados, Panorama e Municípios
ONS, geração por fonte	Mostrar quanto a MMGD e a solar estão gerando no país e comparar com a potência instalada da ANEEL	Geração agora (ONS)
API do Radar Solar	Servir os indicadores municipais por HTTP para o app e, no TP4, para a LLM	Consulta via API

2. Dicionário de dados

Colunas que leio do arquivo da ANEEL:

Coluna	Tipo	Para que uso
`SigUF`	string	Agregação por estado
`NomRegiao`	string	Comparar o município com a região dele
`CodMunicipioIbge`	int64	Chave de junção com o IBGE
`NomMunicipio`	string	Exibição no painel
`DscClasseConsumo`	string	Perfil de quem gera (Residencial, Comercial, Industrial, Rural e Serviço Público)
`DscFonteGeracao`	string	Separar a geração solar das outras fontes
`DscPorte`	string	Diferenciar microgeração de minigeração
`MdaPotenciaInstaladaKW`	double	Métrica principal, potência instalada em kW
`DthAtualizaCadastralEmpreend`	date32	Verificar a defasagem do dado

Colunas que uso do IBGE:

Coluna	Tipo	Para que uso
`id`	int64	Código do município, chave de junção
`nome`	string	Nome oficial
`microrregiao.mesorregiao.UF.sigla`	string	UF de referência

Colunas do arquivo de notícias (`data/external/noticias_aneel.csv`):

Coluna	Tipo	Para que uso
`data`	date	Filtro de período e gráfico de notícias por mês
`categoria`	string	Filtro e gráfico por categoria. Normalizada, porque o site às vezes escreve "Tarifas" e às vezes "TARIFAS"
`titulo`	string	Exibição e nuvem de palavras
`descricao`	string	Resumo exibido na lista de notícias
`url`	string	Link para a notícia original. Também serve para tirar duplicadas
`texto`	string	Texto completo, base da nuvem de palavras e da busca
`tema_gd`	bool	Marca as notícias que citam geração distribuída, micro ou minigeração, solar ou fotovoltaica
`qtd_palavras`	int	Estatística de tamanho das notícias
`coletado_em`	string	Data e hora da coleta, mostrada na tela

O `data/external/noticias_aneel.txt` tem o mesmo conteúdo só em texto (título, resumo e corpo), uma notícia por linha.

Colunas do exemplo de população (`Sample_Data/exemplo_upload_populacao.csv`):

Coluna	Tipo	Para que uso
`CodMunicipioIbge`	int64	Chave de junção com a base da ANEEL
`populacao`	int64	Divisor do indicador kW por mil habitantes
`ano`	int64	Ano da estimativa. Como é igual em todas as linhas, o painel não transforma essa coluna em indicador

Colunas da geração do ONS (`data/external/geracao_ons.csv`, TP3):

Coluna	Tipo	Para que uso
`fonte`	string	Nome da fonte (Hidráulica, Térmica, Nuclear, Eólica, Solar, MMGD, Importação)
`geracao_mw`	float	Geração da fonte no momento da coleta, em MW
`participacao_pct`	float	Participação da fonte na geração total, calculada no script
`atualizacao_ons`	string	Data e hora que o ONS informa na página, para saber de quando é o número
`coletado_em`	string	Data e hora em que o script rodou

Campos devolvidos pela API (TP3): as rotas de consulta devolvem as colunas de `cobertura_municipios.csv` (`CodMunicipioIbge`, `municipio`, `uf`, `NomRegiao`, `qtd_empreendimentos`, `potencia_total_kw`, `potencia_media_kw`). A rota POST recebe `cod_ibge` (int, 7 dígitos) e `populacao` (int maior que zero) e devolve

`kw_por_mil_habitantes` (float). Os tipos são validados pela própria FastAPI: um valor fora do esperado devolve erro 422 com a explicação.

Regras do arquivo enviado pelo usuário: a coluna do código IBGE pode se chamar `CodMunicipioIbge`, `cod_ibge`, `codigo_ibge`, `cod_municipio`, `ibge` ou `codigo`. O separador pode ser vírgula ou ponto e vírgula, e a codificação UTF-8 ou Latin-1, porque é assim que o Excel em português salva CSV. Números no formato brasileiro (1.234,5) são convertidos. Linhas com código inválido ou repetido são descartadas, com aviso na tela.

3. Colunas descartadas por privacidade

O arquivo original tem `NomTitularEmpreendimento`, `NumCPFCNPJ` e `CodCEP`. São dados pessoais, identificam o dono de cada instalação e não servem para nada na pergunta do projeto, que é territorial e agregada.

Optei por não ler essas colunas já na leitura do Parquet, em vez de apagar depois. Assim elas nem chegam a entrar na memória nem nos arquivos gerados.

4. O que encontrei na inspeção

A potência instalada total soma 51.630 MW, e 99,41% disso é radiação solar. Isso confirmou que tratar o projeto como painel solar é fiel ao dado.

Dos 5.571 municípios do IBGE, só 4 não têm nenhum empreendimento registrado. Isso derrubou a minha hipótese inicial, que era mapear vazios de adoção.

A desigualdade aparece na intensidade. Os 10% maiores municípios concentram 58,7% da potência nacional. A mediana municipal é 3.041 kW e o percentil 90 chega a 19.680 kW, uma diferença de mais de seis vezes. Foi esse recorte que passou a orientar o projeto.

Nas notícias, a categoria mais frequente é Tarifas (18 notícias), seguida de Geração (10), Institucional (8) e Fiscalização (8). Os termos que mais aparecem, tirando as stopwords, são pública (108 vezes, de consulta e audiência pública), transmissão (87), consulta (75) e distribuição (68). A pauta recente da agência é tarifária e de rede. Só 5 das 83 notícias tratam de geração distribuída ou solar, uma delas sobre novas regras para preparar a rede para geração solar, baterias e veículos elétricos.

Com a população, o Brasil tem 241 kW de geração distribuída por mil habitantes. Entre os estados a diferença é grande: Mato Grosso tem 806 kW por mil habitantes e o Amazonas tem 75, mais de dez vezes menos. No ranking por habitante aparecem no topo municípios pequenos com usinas de minigeração, como Santa Clara d'Oeste (SP), com 2.710 habitantes. Esse indicador relativo complementa o absoluto, que favorece as capitais.

No ONS, a coleta das 18h17 de 19/10/2026 mostra a hidráulica com 70,1% da geração, a térmica com 15,0% e a eólica com 13,1%. A MMGD aparecia com 0,22% e a solar centralizada com 0,07%. Os números baixos de solar vêm da hora: era fim de tarde. Comparando com a base da ANEEL, a MMGD gerava naquele momento perto de 0,4% da potência instalada de GD. Para tirar conclusão sobre a contribuição real da GD é preciso coletar em vários horários, o que fica planejado para o TP4.

5. Qualidade e limitações

As colunas `CodSubGrupoTarifario` e `SigModalidadeEmpreendimento` vêm todas nulas no arquivo, então não dá para usar.

A ANEEL informa que a atualização da base ficou suspensa entre 23/09/2025 e 13/11/2025 por migração do sistema SISGD, e que segue mais lenta desde então. Por isso o painel sempre mostra a data de referência do dado carregado.

Alguns municípios mais novos vêm do IBGE com `microrregiao` nula. Nesses casos eu pego a UF pelo caminho da `regiao-imediata`, senão o processamento quebra.

As notícias têm uma limitação importante. Durante o período eleitoral o site da ANEEL deixa públicas só as notícias recentes, e o arquivo por ano pede login. Por isso 76 das 83 notícias são de julho a setembro de 2026. Depois do período eleitoral vale rodar o scraping de novo.

O scraping depende do HTML do site. Se o layout mudar, o script para com erro em vez de gravar um arquivo vazio, e o painel continua com a última coleta versionada.

A estimativa de população do IBGE é uma projeção, não um censo. Para comparar municípios ela serve bem, mas não é contagem exata.

O arquivo enviado pelo usuário não passa por checagem de conteúdo além do formato. Se ele subir um número errado, o painel calcula o indicador com esse número.

A coleta do ONS é uma fotografia de um instante, não uma série. O script lê o texto da página procurando cada fonte seguida de um valor em MW, então se o ONS mudar o texto do quadro o script para de achar os números e para com erro.

Os 4 municípios do Amazonas sem nenhum empreendimento vêm com região e potência média vazias no processado. No app isso não atrapalha, mas na API valor vazio não vira JSON e a rota devolvia erro 500. Na carga da API eu preencho a região pela UF e a média com zero.

6. Arquivos gerados

Arquivo	Conteúdo	Tamanho
`data/processed/gd_por_municipio.csv`	Uma linha por município com quantidade e potência	268 KB
`data/processed/cobertura_municipios.csv`	Os 5.571 municípios do IBGE com os indicadores preenchidos	355 KB
`data/processed/gd_por_fonte.csv`	Potência por fonte de geração	546 B
`data/processed/gd_por_classe.csv`	Potência por classe de consumo e UF	5,8 KB
`Sample_Data/amostra_gd_bruto.csv`	500 linhas do dado bruto	42 KB
`Sample_Data/amostra_gd_municipio.csv`	200 municípios do agregado	10 KB
`data/external/noticias_aneel.csv`	83 notícias da ANEEL com texto completo (TP2)	178 KB
`data/external/noticias_aneel.txt`	Texto das notícias, uma por linha, para a nuvem de palavras (TP2)	161 KB
`Sample_Data/exemplo_upload_populacao.csv`	População 2026 dos 5.571 municípios, exemplo de upload (TP2)	106 KB
`data/external/geracao_ons.csv`	Geração do SIN por fonte, coletada com Selenium (TP3)	menos de 1 KB
`data/external/geracao_ons.txt`	Mesma geração em texto, uma fonte por linha (TP3)	menos de 1 KB

Código novo do TP3

O código completo está no repositório. Aqui ficam os arquivos criados no TP3.

Code/API/api.py

```
# API do Radar Solar Municipal, feita com FastAPI.
# Serve os dados de geracao distribuida por municipio que ja estao processados
# em data/processed. O app Streamlit consome essa API na pagina "API".
#
# Pra rodar a API:
#   uvicorn Code.API.api:app --reload --port 8000
# A documentacao automatica (Swagger) fica em http://localhost:8000/docs
#
# Por que uma API separada se os dados ja estao em CSV? Pra separar o acesso ao
# dado da interface. Com a API, o painel (ou qualquer outro programa, ou outra
# pessoa) pega os dados por HTTP sem precisar conhecer os arquivos. E o passo que
# prepara o projeto pro TP4, quando a parte de LLM vai responder perguntas
# consultando esses mesmos endpoints.

import os

import pandas as pd
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel, Field

RAIZ = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
PASTA_PROCESSED = os.path.join(RAIZ, "data", "processed")
PASTA_AMOSTRA = os.path.join(RAIZ, "Sample_Data")

app = FastAPI(
    title="Radar Solar Municipal - API",
    description=(
        "API de consulta da geração distribuída de energia por município, a "
        "partir dos dados abertos da ANEEL. Projeto de Bloco de IA Aplicada, Infnet."
    ),
    version="1.0.0 (TP3)",
)


# ---- carga dos dados (uma vez, quando a API sobe)

def carregar_municipios():
    # mesma logica do app: tenta o processado, cai pra amostra
    caminho = os.path.join(PASTA_PROCESSED, "cobertura_municipios.csv")
    if os.path.exists(caminho):
        return pd.read_csv(caminho)
    caminho_amostra = os.path.join(PASTA_AMOSTRA, "amostra_gd_municipio.csv")
    if os.path.exists(caminho_amostra):
        dados = pd.read_csv(caminho_amostra)
        return dados.rename(columns={"NomMunicipio": "municipio", "SigUF": "uf"})
    return pd.DataFrame()


MUNICIPIOS = carregar_municipios()

# os 4 municipios sem nenhum empreendimento (todos no AM) vem com regiao e
# media vazias. NaN nao vira JSON e a API dava erro 500 neles, descobri testando
# /municipios/1300060. preencho a regiao pela UF e a media com zero
if not MUNICIPIOS.empty:
    regiao_da_uf = MUNICIPIOS.dropna(subset=["NomRegiao"]).groupby("uf")["NomRegiao"].first()
    MUNICIPIOS["NomRegiao"] = MUNICIPIOS["NomRegiao"].fillna(MUNICIPIOS["uf"].map(regiao_da_uf))
    MUNICIPIOS["potencia_media_kw"] = MUNICIPIOS["potencia_media_kw"].fillna(0)


# ---- modelos (o FastAPI usa pra validar e documentar sozinho)

class Municipio(BaseModel):
    CodMunicipioIbge: int
    municipio: str
    uf: str
    NomRegiao: str | None = None
    qtd_empreendimentos: int
    potencia_total_kw: float
    potencia_media_kw: float


class PopulacaoEntrada(BaseModel):
    # o que o POST recebe: um municipio e a populacao dele, pra calcular
    # a potencia instalada por mil habitantes
    cod_ibge: int = Field(..., description="Código IBGE do município (7 dígitos)", examples=[3304557])
    populacao: int = Field(..., gt=0, description="População do município", examples=[6211000])


class PopulacaoResposta(BaseModel):
    municipio: str
    uf: str
    potencia_total_kw: float
    populacao: int
    kw_por_mil_habitantes: float


# ---- rotas

@app.get("/", tags=["info"])
def raiz():
    # rota de boas vindas, diz se os dados carregaram
    return {
        "projeto": "Radar Solar Municipal",
        "municipios_carregados": len(MUNICIPIOS),
        "documentacao": "/docs",
        "rotas": ["/municipios", "/municipios/{cod_ibge}", "/ranking", "/indicador-por-habitante"],
    }


@app.get("/municipios", tags=["consulta"])
def listar_municipios(
    uf: str | None = Query(None, description="Filtrar por UF, ex.: RJ", max_length=2),
    limite: int = Query(50, ge=1, le=500, description="Quantos municípios devolver"),
):
    # GET com filtro opcional por UF. ex: /municipios?uf=RJ&limite=10
    if MUNICIPIOS.empty:
        raise HTTPException(status_code=503, detail="Dados não carregados. Rode o processamento.")

    dados = MUNICIPIOS
    if uf:
        dados = dados[dados["uf"].str.upper() == uf.upper()]
        if dados.empty:
            raise HTTPException(status_code=404, detail="Nenhum município para a UF %s" % uf)

    dados = dados.sort_values("potencia_total_kw", ascending=False).head(limite)
    return {
        "total": len(dados),
        "uf": uf.upper() if uf else "Brasil",
        "municipios": dados.to_dict(orient="records"),
    }


@app.get("/municipios/{cod_ibge}", response_model=Municipio, tags=["consulta"])
def municipio_por_codigo(cod_ibge: int):
    # GET de um municipio so, pelo codigo IBGE. ex: /municipios/3304557
    if MUNICIPIOS.empty:
        raise HTTPException(status_code=503, detail="Dados não carregados.")
    linha = MUNICIPIOS[MUNICIPIOS["CodMunicipioIbge"] == cod_ibge]
    if linha.empty:
        raise HTTPException(status_code=404, detail="Município %d não encontrado" % cod_ibge)
```

```
    return linha.iloc[0].to_dict()


@app.get("/ranking", tags=["consulta"])
def ranking(
    por: str = Query("potencia", description="potencia, quantidade ou media"),
    uf: str | None = Query(None, max_length=2),
    top: int = Query(10, ge=1, le=100),
):
    # GET do ranking configuravel. ex: /ranking?por=potencia&uf=SP&top=5
    if MUNICIPIOS.empty:
        raise HTTPException(status_code=503, detail="Dados não carregados.")

    colunas = {
        "potencia": "potencia_total_kw",
        "quantidade": "qtd_empreendimentos",
        "media": "potencia_media_kw",
    }
    if por not in colunas:
        raise HTTPException(status_code=400, detail="Use por = potencia, quantidade ou media")

    dados = MUNICIPIOS
    if uf:
        dados = dados[dados["uf"].str.upper() == uf.upper()]

    coluna = colunas[por]
    dados = dados.sort_values(coluna, ascending=False).head(top)
    return {
        "criterio": por,
        "uf": uf.upper() if uf else "Brasil",
        "ranking": dados[["municipio", "uf", coluna]].to_dict(orient="records"),
    }


@app.post("/indicador-por-habitante", response_model=PopulacaoResposta, tags=["cálculo"])
def indicador_por_habitante(entrada: PopulacaoEntrada):
    # POST: recebe um municipio e a populacao, devolve a potencia instalada
    # por mil habitantes. ex de corpo: {"cod_ibge": 3304557, "populacao": 6211000}
    if MUNICIPIOS.empty:
        raise HTTPException(status_code=503, detail="Dados não carregados.")

    linha = MUNICIPIOS[MUNICIPIOS["CodMunicipioIbge"] == entrada.cod_ibge]
    if linha.empty:
        raise HTTPException(status_code=404, detail="Município %d não encontrado" % entrada.cod_ibge)

    linha = linha.iloc[0]
    kw_por_mil = linha["potencia_total_kw"] / entrada.populacao * 1000
    return {
        "municipio": linha["municipio"],
        "uf": linha["uf"],
        "potencia_total_kw": round(float(linha["potencia_total_kw"]), 2),
        "populacao": entrada.populacao,
        "kw_por_mil_habitantes": round(kw_por_mil, 2),
    }
```

Code/DataAcquisition/scraping_ons_selenium.py

```
# Scraping da pagina "Energia Agora" do ONS com Selenium.
# Rodar separado, igual aos outros: python Code/DataAcquisition/scraping_ons_selenium.py
# Salva em data/external/geracao_ons.csv e .txt
#
# Por que Selenium e nao Beautiful Soup aqui? Essa pagina do ONS carrega os
# numeros de geracao por JavaScript depois que a pagina abre. Se eu baixar o
# HTML com requests + BeautifulSoup vem a estrutura da pagina mas os valores
# ficam vazios (a pagina tem 154 KB de HTML e so 2 KB de texto util antes do JS
# rodar). Com o Selenium eu abro a pagina num navegador de verdade, espero o JS
# montar os numeros e so entao leio a tela.
#
# O dado e a geracao nacional por fonte em tempo real. Interessa pro projeto
# porque mostra onde a solar e a MMGD (micro e minigeracao distribuida, que e o
# tema do painel) entram na matriz do pais inteiro.

import os
import re
import sys
import time
from datetime import datetime

import pandas as pd
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By

RAIZ = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
PASTA_EXTERNAL = os.path.join(RAIZ, "data", "external")

URL_ONS = "https://www.ons.org.br/paginas/energia-agora/carga-e-geracao"

# o chrome e o chromedriver podem estar em lugares diferentes dependendo da
# maquina. deixo o padrao do sistema (PATH) e dou pra sobrescrever por variavel
# de ambiente pra rodar no Streamlit Cloud, que instala o chromium via packages.txt
CHROME_BIN = os.environ.get("CHROME_BIN")
CHROMEDRIVER = os.environ.get("CHROMEDRIVER")

# as fontes que aparecem no quadro "Geracao (MW)" do ONS
FONTES = ["Hidráulica", "Térmica", "Nuclear", "Eólica", "Solar", "MMGD", "Importação"]


def abrir_navegador():
    opcoes = Options()
    if CHROME_BIN:
        opcoes.binary_location = CHROME_BIN
    # headless = sem abrir janela, roda no servidor. os outros flags sao o
    # combo padrao pra rodar chrome dentro de container/linux sem tela
    for flag in ["--headless=new", "--no-sandbox", "--disable-dev-shm-usage",
                 "--disable-gpu", "--window-size=1920,1080"]:
        opcoes.add_argument(flag)
    opcoes.add_argument("--user-agent=Mozilla/5.0 (projeto academico Infnet - Radar Solar Municipal)")

    servico = Service(CHROMEDRIVER) if CHROMEDRIVER else Service()
    navegador = webdriver.Chrome(service=servico, options=opcoes)
    navegador.set_page_load_timeout(60)
    return navegador


def ler_geracao(texto_pagina):
    # o ONS escreve cada fonte como "Ger. Solar: 65,5 MW". pego o numero que
    # vem depois de cada nome de fonte
    registros = []
    for fonte in FONTES:
        # aceita "Ger. Solar: 65,5 MW" e "Solar: 65,5 MW"
        padrao = re.compile(r"%s\s*:?\s*([\d.]+,\d+|\d+)\s*MW" % re.escape(fonte), re.IGNORECASE)
        achado = padrao.search(texto_pagina)
        if achado:
            # numero vem em formato brasileiro (63.464,8), converto pra float
            valor = float(achado.group(1).replace(".", "").replace(",", "."))
            registros.append({"fonte": fonte, "geracao_mw": valor})
    return registros


def ler_atualizacao(texto_pagina):
    # a pagina mostra "ULTIMA ATUALIZACAO: 19/10/2026 18:16"
    achado = re.search(r"ATUALIZA[ÇC][ÃA]O:\s*([\d/]+\s+[\d:]+)", texto_pagina, re.IGNORECASE)
    return achado.group(1) if achado else ""


def coletar():
    navegador = abrir_navegador()
```

```
    try:
        print("Abrindo a pagina do ONS...")
        navegador.get(URL_ONS)

        # o JS precisa de um tempo pra buscar os numeros e montar a tela.
        # espero ate aparecer algum valor em MW, checando de 2 em 2 segundos
        texto = ""
        for tentativa in range(10):
            time.sleep(2)
            texto = navegador.find_element(By.TAG_NAME, "body").text
            if "MW" in texto and ler_geracao(texto):
                print("Numeros carregaram depois de ~%d segundos" % ((tentativa + 1) * 2))
                break
        else:
            raise RuntimeError("Os numeros nao carregaram, o layout do ONS pode ter mudado")

        geracao = ler_geracao(texto)
        atualizacao = ler_atualizacao(texto)
        return geracao, atualizacao
    finally:
        navegador.quit()


if __name__ == "__main__":
    try:
        geracao, atualizacao = coletar()
    except Exception as erro:
        sys.exit("Falhou o scraping do ONS: %s" % erro)

    if not geracao:
        sys.exit("Nenhum valor de geracao encontrado")

    tabela = pd.DataFrame(geracao)
    total = tabela["geracao_mw"].sum()
    tabela["participacao_pct"] = (tabela["geracao_mw"] / total * 100).round(2)
    tabela["atualizacao_ons"] = atualizacao
    tabela["coletado_em"] = datetime.now().strftime("%Y-%m-%d %H:%M")

    os.makedirs(PASTA_EXTERNAL, exist_ok=True)
    caminho_csv = os.path.join(PASTA_EXTERNAL, "geracao_ons.csv")
    tabela.to_csv(caminho_csv, index=False, encoding="utf-8")

    # um txt curtinho so pra deixar registrado em texto, no mesmo padrao do
    # scraping de noticias
    caminho_txt = os.path.join(PASTA_EXTERNAL, "geracao_ons.txt")
    with open(caminho_txt, "w", encoding="utf-8") as arquivo:
        arquivo.write("Geracao por fonte no SIN - ONS (coletado via Selenium)\n")
        arquivo.write("Atualizacao do ONS: %s\n\n" % atualizacao)
        for _, linha in tabela.iterrows():
            arquivo.write("%s: %.1f MW (%.2f%%)\n" % (linha["fonte"], linha["geracao_mw"], linha["participacao_pct"]))

    print("Salvei %d fontes em %s" % (len(tabela), caminho_csv))
    print("Geracao total do SIN: %.0f MW" % total)
    solar_mmgd = tabela[tabela["fonte"].isin(["Solar", "MMGD"])]["geracao_mw"].sum()
    print("Solar + MMGD: %.1f MW (%.2f%% da matriz)" % (solar_mmgd, solar_mmgd / total * 100))
```

app.py

```
# Dashboard do Radar Solar Municipal. pra rodar: streamlit run app.py
# TP1: painel demo com os dados da ANEEL
# TP2: filtros, exploracao por municipio, noticias da ANEEL (scraping),
#      upload/download de CSV, cache e estado de sessao
# TP3: virou app multipagina (pasta paginas/), com menu de navegacao, pagina de
#      geracao do ONS coletada com Selenium e pagina que consome a API FastAPI
#
# Esse arquivo agora e so a "casca": configura a pagina, monta a barra lateral
# com os filtros (que valem pra todas as paginas) e chama a pagina escolhida.
# O conteudo de cada pagina fica em paginas/, e o codigo comum em Code/App/comum.py

import os
import sys

import streamlit as st

RAIZ = os.path.dirname(os.path.abspath(__file__))
sys.path.append(os.path.join(RAIZ, "Code", "App"))
import comum  # noqa: E402

st.set_page_config(page_title="Radar Solar Municipal", page_icon="☀", layout="wide")


# ---- estado de sessao

# tudo que precisa sobreviver entre um clique e outro, e entre uma pagina e
# outra, fica no session_state. o setdefault so cria na primeira execucao
st.session_state.setdefault("dados_usuario", None)
st.session_state.setdefault("arquivo_enviado", None)
st.session_state.setdefault("avisos_upload", [])
st.session_state.setdefault("historico", [])
st.session_state.setdefault("f_regioes", [])
st.session_state.setdefault("f_ufs", [])
st.session_state.setdefault("consultas_api", 0)


def limpar_filtros():
    st.session_state["f_regioes"] = []
    st.session_state["f_ufs"] = []


# sobe a API do FastAPI junto com o app se ela ainda nao estiver rodando
comum.iniciar_api_local()


# ---- menu de navegacao

# st.navigation monta o menu na barra lateral. separei as paginas em secoes
# pra ficar claro o que e analise, o que e coleta e o que e o projeto
paginas = {
    "Análise": [
        st.Page("paginas/1_panorama.py", title="Panorama", icon="📊", default=True),
        st.Page("paginas/2_municipios.py", title="Municípios", icon="🏙"),
    ],
    "Dados coletados": [
        st.Page("paginas/3_noticias.py", title="Notícias ANEEL", icon="📰"),
        st.Page("paginas/4_geracao_ons.py", title="Geração agora (ONS)", icon="⚡"),
        st.Page("paginas/5_api.py", title="Consulta via API", icon="🔌"),
        st.Page("paginas/6_seus_dados.py", title="Seus dados", icon="📂"),
    ],
    "Projeto": [
        st.Page("paginas/7_sobre.py", title="Sobre o projeto", icon="ℹ"),
        st.Page("paginas/8_proximos_passos.py", title="Próximos passos: IA", icon=":material/smart_toy:"),
    ],
}
navegacao = st.navigation(paginas)


# ---- barra lateral: filtros (aparecem em todas as paginas)

base, municipios, origem, _, _ = comum.montar_base()

with st.sidebar:
    st.header("⚙ Filtros")
    st.caption("Valem para Panorama e Municípios e ficam guardados enquanto a sessão estiver aberta.")

    if base is not None:
        regioes = sorted(base["NomRegiao"].dropna().unique())
        st.multiselect("Região", regioes, key="f_regioes", placeholder="Todas")

        if st.session_state["f_regioes"]:
```

```
            ufs_possiveis = sorted(base.loc[base["NomRegiao"].isin(st.session_state["f_regioes"]), "uf"].unique())
        else:
            ufs_possiveis = sorted(base["uf"].dropna().unique())

        # se a pessoa tira uma regiao, as UFs dela saem da selecao senao o
        # multiselect reclama de valor que nao esta nas opcoes
        st.session_state["f_ufs"] = [u for u in st.session_state["f_ufs"] if u in ufs_possiveis]
        st.multiselect("UF", ufs_possiveis, key="f_ufs", placeholder="Todas")

        st.button("Limpar filtros", on_click=limpar_filtros, width="stretch")

    st.divider()
    st.subheader("Sua sessão")
    if st.session_state["arquivo_enviado"]:
        st.write("📎 Arquivo carregado: **%s**" % st.session_state["arquivo_enviado"])
    else:
        st.write("Nenhum arquivo enviado.")
    if st.session_state["consultas_api"]:
        st.write("🔎 Consultas feitas à API: %d" % st.session_state["consultas_api"])
    if st.session_state["historico"]:
        st.write("Municípios consultados:")
        for item in st.session_state["historico"]:
            st.caption("• " + item)


# ---- cabecalho comum

st.title("☀ Radar Solar Municipal")
st.markdown(
    "Geração distribuída de energia no Brasil por município · "
    "**Projeto de Bloco: IA Aplicada**, Instituto Infnet · Rafael Celestino Tomiazi"
)

if base is None:
    st.error(
        "Não encontrei os dados processados nem a amostra. "
        "Rode `python Code/DataAcquisition/coleta_dados.py` e depois "
        "`python Code/DataPreparation/processa_dados.py`."
    )
    st.stop()

data_referencia, origem_dado = comum.consultar_metadados()
if origem_dado.startswith("API"):
    st.success("Base da ANEEL atualizada em %s. Fonte: %s." % (data_referencia, origem_dado))
elif data_referencia:
    st.warning("Base da ANEEL de %s. Fonte: %s." % (data_referencia, origem_dado))
else:
    st.error("Não consegui confirmar a data da base. Motivo: %s." % origem_dado)

if origem == "amostra":
    st.warning("Exibindo a amostra versionada. Para o painel completo, rode os scripts do README.")

navegacao.run()

st.divider()
st.caption(
    "Dados: ANEEL (licença ODbL), IBGE e ONS. Notícias: gov.br/aneel. "
    "Ferramenta de IA usada no desenvolvimento: Claude (Anthropic). TP3 da disciplina."
)
```

Code/App/comum.py

```
# Coisas que todas as paginas usam: carga dos dados com cache, montagem da base
# com o upload do usuario, filtros e o grafico de barras.
# No TP2 isso tudo morava no app.py. No TP3 o app virou multipagina, e cada
# pagina e um arquivo separado em paginas/, entao separei o que e comum aqui pra
# nao repetir o mesmo codigo em sete arquivos.

import os
import sys
import threading

import altair as alt
import numpy as np
import pandas as pd
import requests
import streamlit as st

RAIZ = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
PASTA_PROCESSED = os.path.join(RAIZ, "data", "processed")
PASTA_EXTERNAL = os.path.join(RAIZ, "data", "external")
PASTA_AMOSTRA = os.path.join(RAIZ, "Sample_Data")

sys.path.append(RAIZ)  # pro "from Code.API.api import app" funcionar
sys.path.append(os.path.join(RAIZ, "Code", "DataAcquisition"))
sys.path.append(os.path.join(RAIZ, "Code", "App"))
from fonte_dados import buscar_metadados  # noqa: E402
import dados_usuario  # noqa: E402
import textos  # noqa: E402

# endereco da API do FastAPI. da pra trocar por variavel de ambiente se a API
# estiver rodando em outro lugar
API_URL = os.environ.get("RADAR_API_URL", "http://localhost:8000")


# ---- carga dos dados (tudo com cache)

@st.cache_data
def carregar(nome_arquivo, pasta=PASTA_PROCESSED):
    # sem o cache o streamlit releria os CSV do disco a cada clique
    caminho = os.path.join(pasta, nome_arquivo)
    try:
        return pd.read_csv(caminho)
    except FileNotFoundError:
        return None


def carregar_municipios():
    # se o processado nao existir cai pra amostra versionada
    dados = carregar("cobertura_municipios.csv")
    if dados is not None:
        return dados, "completo"
    dados = carregar("amostra_gd_municipio.csv", PASTA_AMOSTRA)
    if dados is not None:
        dados = dados.rename(columns={"NomMunicipio": "municipio", "SigUF": "uf"})
        return dados, "amostra"
    return None, "ausente"


@st.cache_data(ttl=3600)
def consultar_metadados():
    # a base nao muda de minuto em minuto, 1h de cache ja ta bom
    return buscar_metadados()


@st.cache_data
def carregar_noticias():
    caminho = os.path.join(PASTA_EXTERNAL, "noticias_aneel.csv")
    try:
        noticias = pd.read_csv(caminho, parse_dates=["data"])
    except FileNotFoundError:
        return None
    noticias["texto"] = noticias["texto"].fillna("")
    noticias["descricao"] = noticias["descricao"].fillna("")
    return noticias


@st.cache_data
def carregar_geracao_ons():
```

```
    # CSV gerado pelo scraping com Selenium (scraping_ons_selenium.py)
    return carregar("geracao_ons.csv", PASTA_EXTERNAL)


@st.cache_data(show_spinner="Gerando a nuvem de palavras...")
def nuvem_e_termos(lista_textos, extras):
    # recebe tupla (e nao lista) pq o cache precisa de argumento hashable
    contagem = textos.contar_termos(lista_textos, extras)
    if not contagem:
        return None, pd.DataFrame(columns=["termo", "ocorrencias"])
    imagem = textos.gerar_nuvem(dict(contagem.most_common(200)))
    termos = pd.DataFrame(contagem.most_common(20), columns=["termo", "ocorrencias"])
    return imagem, termos


def grafico_barras(serie, titulo_valor, horizontal=True):
    # o st.bar_chart ordena pelo nome, entao fiz com altair pra controlar a ordem
    tabela = serie.rename("valor").rename_axis("item").reset_index()
    eixo_item = alt.Y("item:N", sort="-x", title=None, axis=alt.Axis(labelOverlap=False, labelLimit=220)) if horizontal else alt.X("item:N", sort="-y", title=None)
    eixo_valor = alt.X("valor:Q", title=titulo_valor) if horizontal else alt.Y("valor:Q", title=titulo_valor)
    grafico = (
        alt.Chart(tabela)
        .mark_bar(color="#f59e0b")
        .encode(eixo_item, eixo_valor, tooltip=["item", alt.Tooltip("valor:Q", format=",.1f")])
    )
    altura = max(200, 26 * len(tabela)) if horizontal else 320
    st.altair_chart(grafico.properties(height=altura), width="stretch")


# ---- base com o upload do usuario

def montar_base():
    # devolve (base, municipios, origem, colunas_usuario, indicadores).
    # roda em toda pagina, mas e barato pq a leitura do CSV ta em cache
    municipios, origem = carregar_municipios()
    if municipios is None:
        return None, None, origem, [], {}

    if st.session_state.get("dados_usuario") is not None:
        base = dados_usuario.juntar(municipios, st.session_state["dados_usuario"])
        colunas_usuario = [c for c in st.session_state["dados_usuario"].columns if c != dados_usuario.COLUNA_CHAVE]
    else:
        base = municipios.copy()
        colunas_usuario = []

    base["rotulo"] = base["municipio"] + " - " + base["uf"]
    base["potencia_mw"] = base["potencia_total_kw"] / 1000

    indicadores = {
        "Potência instalada (MW)": "potencia_mw",
        "Número de empreendimentos": "qtd_empreendimentos",
        "Potência média por empreendimento (kW)": "potencia_media_kw",
    }
    for coluna in colunas_usuario:
        # coluna constante (tipo "ano" = 2026 em todas as linhas) nao vira indicador
        if pd.api.types.is_numeric_dtype(base[coluna]) and base[coluna].nunique() > 1:
            nome = "kW por mil de '%s' (seu arquivo)" % coluna
            divisor = base[coluna].replace(0, np.nan)
            base["kw_por_mil_" + coluna] = base["potencia_total_kw"] / divisor * 1000
            indicadores[nome] = "kw_por_mil_" + coluna

    return base, municipios, origem, colunas_usuario, indicadores


def aplicar_filtros(base):
    # os filtros moram na barra lateral do app.py e ficam no session_state,
    # entao qualquer pagina consegue ler sem repetir os widgets
    regioes = st.session_state.get("f_regioes", [])
    ufs = st.session_state.get("f_ufs", [])
    filtro = pd.Series(True, index=base.index)
    if regioes:
        filtro &= base["NomRegiao"].isin(regioes)
    if ufs:
        filtro &= base["uf"].isin(ufs)

    if ufs:
        recorte = ", ".join(ufs)
    elif regioes:
        recorte = ", ".join(regioes)
    else:
        recorte = "Brasil"
    return base[filtro], recorte


# ---- API do FastAPI

def api_no_ar():
    try:
        return requests.get(API_URL + "/", timeout=2).ok
    except requests.RequestException:
        return False


@st.cache_resource
def iniciar_api_local():
    # se ninguem subiu a API antes, o proprio app sobe ela numa thread separada.
    # assim quem baixar o projeto so precisa rodar o streamlit, sem abrir um
    # segundo terminal. cache_resource garante que isso acontece uma vez so,
    # e nao a cada clique
    if api_no_ar():
        return "ja estava no ar"
    if "localhost" not in API_URL and "127.0.0.1" not in API_URL:
        return "API externa fora do ar"

    import uvicorn
    from Code.API.api import app as app_api

    porta = int(API_URL.rsplit(":", 1)[-1])
    servidor = uvicorn.Server(uvicorn.Config(app_api, host="127.0.0.1", port=porta, log_level="warning"))
    threading.Thread(target=servidor.run, daemon=True).start()
    return "iniciada pelo app"


def chamar_api(metodo, rota, **kwargs):
    # devolve (json, erro). a pagina mostra o erro na tela em vez de quebrar
    try:
        resposta = requests.request(metodo, API_URL + rota, timeout=10, **kwargs)
    except requests.RequestException as erro:
        return None, "Não consegui falar com a API em %s (%s)" % (API_URL, erro.__class__.__name__)
    if not resposta.ok:
        try:
            detalhe = resposta.json().get("detail")
        except ValueError:
            detalhe = resposta.text
        return None, "A API respondeu %d: %s" % (resposta.status_code, detalhe)
    return resposta.json(), None
```

paginas/5_api.py

```
# Pagina Consulta via API: o app conversando com a API FastAPI (novo no TP3).
# Aqui o painel nao le o CSV direto, ele pede os dados pra API por HTTP, do
# mesmo jeito que qualquer outro programa faria.

import pandas as pd
import streamlit as st

import comum
```

```
def contar_consulta():
    st.session_state["consultas_api"] += 1


st.header("Consulta via API")
st.write(
    "Esta página não lê os arquivos do projeto: ela pede os dados para a API do Radar "
    "Solar, feita com **FastAPI**. O código da API está em `Code/API/api.py` e a "
    "documentação interativa fica em `/docs` no endereço da API."
)

if comum.api_no_ar():
    st.success("API no ar em %s · [abrir documentação (Swagger)](%s/docs)" % (comum.API_URL, comum.API_URL))
else:
    st.error(
        "A API não respondeu em %s. Suba ela com "
        "`uvicorn Code.API.api:app --port 8000` e recarregue a página." % comum.API_URL
    )
    st.stop()

with st.expander("Rotas disponíveis"):
    st.markdown(
        """
| Método | Rota | O que faz |
|---|---|---|
| GET | `/` | Status da API e quantos municípios foram carregados |
| GET | `/municipios?uf=RJ&limite=50` | Lista municípios, com filtro opcional por UF |
| GET | `/municipios/{cod_ibge}` | Ficha de um município pelo código IBGE |
| GET | `/ranking?por=potencia&uf=SP&top=10` | Ranking por potência, quantidade ou potência média |
| POST | `/indicador-por-habitante` | Recebe código IBGE e população, devolve kW por mil habitantes |
        """
    )

aba_ranking, aba_ficha, aba_post = st.tabs(["GET /ranking", "GET /municipios/{cod_ibge}", "POST /indicador-por-habitante"])

# ---- GET ranking

with aba_ranking:
    c1, c2, c3 = st.columns(3)
    criterio = c1.selectbox("Critério", ["potencia", "quantidade", "media"], key="api_criterio")
    uf = c2.text_input("UF (vazio = Brasil)", max_chars=2, key="api_uf").strip().upper()
    top = c3.slider("Top", 3, 30, 10, key="api_top")

    parametros = {"por": criterio, "top": top}
    if uf:
        parametros["uf"] = uf

    if st.button("Consultar ranking", key="btn_ranking", on_click=contar_consulta):
        resposta, erro = comum.chamar_api("GET", "/ranking", params=parametros)
        if erro:
            st.error(erro)
        else:
            tabela = pd.DataFrame(resposta["ranking"])
            st.write("Ranking por **%s** · %s" % (resposta["criterio"], resposta["uf"]))
            coluna_valor = [c for c in tabela.columns if c not in ("municipio", "uf")][0]
            comum.grafico_barras(tabela.set_index("municipio")[coluna_valor], coluna_valor)
            with st.expander("Resposta JSON da API"):
                st.json(resposta)

# ---- GET municipio

with aba_ficha:
    base, *_ = comum.montar_base()
    rotulos = base.sort_values("potencia_total_kw", ascending=False)[["rotulo", "CodMunicipioIbge"]]
    escolhido = st.selectbox("Município", rotulos["rotulo"].tolist(), key="api_municipio")
    cod = int(rotulos.loc[rotulos["rotulo"] == escolhido, "CodMunicipioIbge"].iloc[0])
    st.caption("Chamada: `GET %s/municipios/%d`" % (comum.API_URL, cod))

    if st.button("Buscar ficha", key="btn_ficha", on_click=contar_consulta):
        resposta, erro = comum.chamar_api("GET", "/municipios/%d" % cod)
        if erro:
            st.error(erro)
        else:
            f1, f2, f3 = st.columns(3)
            f1.metric("Potência instalada", ("%.2f MW" % (resposta["potencia_total_kw"] / 1000)).replace(".", ","))
            f2.metric("Empreendimentos", "{:,}".format(resposta["qtd_empreendimentos"]).replace(",", "."))
            f3.metric("Média por empreendimento", ("%.1f kW" % resposta["potencia_media_kw"]).replace(".", ","))
            st.json(resposta)

# ---- POST indicador por habitante

with aba_post:
    st.write(
        "Envia para a API o código IBGE de um município e a população, e a API devolve a "
        "potência instalada por mil habitantes. É o mesmo cálculo da página *Seus dados*, "
        "só que feito do lado da API."
    )
    # st.form junta os campos e so manda quando clica, senao cada digito
    # disparava uma chamada
    with st.form("form_post"):
        p1, p2 = st.columns(2)
        cod_post = p1.number_input("Código IBGE", min_value=1000000, max_value=9999999, value=3304557, step=1)
        populacao = p2.number_input("População", min_value=1, value=6211000, step=1000)
        # o contador vai no on_click pq a barra lateral e desenhada antes da
        # pagina. contando aqui dentro o numero aparecia atrasado em um
        enviar = st.form_submit_button("Enviar (POST)", on_click=contar_consulta)

    if enviar:
        corpo = {"cod_ibge": int(cod_post), "populacao": int(populacao)}
        st.caption("Corpo enviado: `%s`" % corpo)
        resposta, erro = comum.chamar_api("POST", "/indicador-por-habitante", json=corpo)
        if erro:
            st.error(erro)
        else:
            st.metric(
                "%s - %s" % (resposta["municipio"], resposta["uf"]),
                ("%.1f kW por mil habitantes" % resposta["kw_por_mil_habitantes"]).replace(".", ","),
            )
            st.json(resposta)
```

paginas/4_geracao_ons.py

```
# Pagina Geracao agora: dado do ONS coletado com Selenium (novo no TP3).
# O painel mostra a GD instalada por municipio. Essa pagina coloca isso em
# perspectiva com o que o pais inteiro esta gerando naquele momento.

import streamlit as st

import comum

st.header("Geração do Sistema Interligado Nacional")
st.write(
    "Quanto cada fonte estava gerando no país no momento da coleta, segundo o "
    "Operador Nacional do Sistema Elétrico (ONS). A **MMGD** é a micro e minigeração "
    "distribuída, o mesmo tipo de empreendimento que o resto do painel mostra por município."
)

geracao = comum.carregar_geracao_ons()

if geracao is None or geracao.empty:
    st.warning(
        "Arquivo `data/external/geracao_ons.csv` não encontrado. Rode "
        "`python Code/DataAcquisition/scraping_ons_selenium.py` para coletar."
    )
    st.stop()

st.caption(
    "Coletado com **Selenium** em %s (atualização do ONS: %s). A página do ONS monta "
    "esses números com JavaScript depois de abrir, por isso o Beautiful Soup sozinho "
```

```
    "não enxerga os valores." % (geracao["coletado_em"].iloc[0], geracao["atualizacao_ons"].iloc[0])
)

total = geracao["geracao_mw"].sum()
renovavel = geracao[geracao["fonte"].isin(["Hidráulica", "Eólica", "Solar", "MMGD"])]["geracao_mw"].sum()
mmgd = geracao.loc[geracao["fonte"] == "MMGD", "geracao_mw"].sum()
solar = geracao.loc[geracao["fonte"] == "Solar", "geracao_mw"].sum()

g1, g2, g3, g4 = st.columns(4)
g1.metric("Geração total", ("%.1f GW" % (total / 1000)).replace(".", ","))
g2.metric("Renovável", ("%.1f%%" % (renovavel / total * 100)).replace(".", ","))
g3.metric("MMGD (distribuída)", ("%.1f MW" % mmgd).replace(".", ","))
g4.metric("Solar centralizada", ("%.1f MW" % solar).replace(".", ","))

st.subheader("Geração por fonte (MW)")
comum.grafico_barras(geracao.set_index("fonte")["geracao_mw"], "MW")

st.dataframe(
    geracao[["fonte", "geracao_mw", "participacao_pct"]],
    hide_index=True,
    width="stretch",
    column_config={
        "fonte": "Fonte",
        "geracao_mw": st.column_config.NumberColumn("Geração (MW)", format="%.1f"),
        "participacao_pct": st.column_config.NumberColumn("Participação (%)", format="%.2f"),
    },
)

# comparacao que liga as duas fontes do projeto: o que esta instalado (ANEEL)
# contra o que esta gerando agora (ONS)
base, *_ = comum.montar_base()
instalada_mw = base["potencia_total_kw"].sum() / 1000
st.subheader("Instalado x gerando agora")
st.write(
    "A base da ANEEL soma **%s MW** de GD instalada. No momento da coleta, a MMGD gerava "
    "**%s MW**, ou %s da capacidade. A diferença vem da hora do dia: painel solar só gera "
    "com sol, e uma coleta feita à noite mostra a MMGD quase zerada. Para o TP4 a ideia é "
    "coletar várias vezes ao longo do dia e ver essa curva."
    % (
        "{:,.0f}".format(instalada_mw).replace(",", "."),
        ("%.1f" % mmgd).replace(".", ","),
        ("%.1f%%" % (mmgd / instalada_mw * 100)).replace(".", ","),
    )
)

st.download_button(
    "⬇ Baixar geração do ONS (CSV)",
    geracao.to_csv(index=False).encode("utf-8"),
    file_name="geracao_ons.csv",
    mime="text/csv",
)
```